ЗВІТ З ЛАБОРАТОРНОГО ПРОГРАМНОГО ПРОЄКТУ №2

Дата: 12.10.2026
Група: К-24

Дослідження паралельних алгоритмів C++23 — Варіант 5: std::min_element

Виконавець: Величко Максим Юрійович (група К-24)
Апаратна платформа: Intel(R) Core(TM) i3-1005G1 (2 фізичні ядра / 4 обчислювальні потоки, 4 MB L3 Cache, 3.40 GHz)
Середовище виконання: GCC 13.2.0 (x86_64-linux-gnu, Ubuntu 24.04), стандарт C++23, бібліотека Intel TBB

`GitHub Repository:` https://github.com/pqxq/proj2

1. ПОСТАНОВКА ЗАДАЧІ ТА АЛГОРИТМІЧНИЙ ОПИС

Метою роботи є експериментальне дослідження швидкодії операції пошуку мінімального елемента (`std::min_element`) на масивах 32-бітних цілих чисел різного обсягу (10^5, 10^6 та 10^7 елементів). У межах дослідження порівнюються: (1) послідовний бібліотечний алгоритм без політики виконання; (2) бібліотечний алгоритм із політиками `std::execution::seq`, `par`, `par_unseq`; (3) власний паралельний алгоритм із розбиттям масиву на K частин одинакової довжини, обробкою кожної частини в окремому потоці `std::jthread` та наступною фінальною редукцією.

Формат даних: Вхідні послідовності згенеровано 64-бітним генератором `std::mt19937_64` (діапазон [−1,000,000; 1,000,000]) з фіксованим сидом і збережено у файли (`data_100k.txt`, `data_1m.txt`, `data_10m.txt`) для гарантування ідентичності умов усіх запусків.

2. ПОРІВНЯЛЬНИЙ АНАЛІЗ БІБЛІОТЕЧНИХ АЛГОРИТМІВ ТА ПОЛІТИК

Усі вимірювання часу наведено в мілісекундах (середнє значення за 5 ітераціями після попереднього "прогріву" кеш-пам'яті) для режимів без оптимізації (`-O0`) та з максимальною оптимізацією (`-O3 -march=native`).

Алгоритм / Політика виконання	10^5 (-O0)	10^5 (-O3)	10^6 (-O0)	10^6 (-O3)	10^7 (-O0)	10^7 (-O3)
`std::min_element` (без політики)	0.024 мс	0.026 мс	0.254 мс	0.230 мс	2.675 мс	2.449 мс
`std::min_element` (std::execution::seq)	**0.016 мс**	**0.020 мс**	**0.146 мс**	**0.171 мс**	**2.909 мс**	**2.056 мс**
`std::min_element` (std::execution::par)	0.185 мс	0.214 мс	0.808 мс	0.834 мс	7.637 мс	9.274 мс
`std::min_element` (std::execution::par_unseq)	0.197 мс	0.195 мс	0.880 мс	0.842 мс	8.642 мс	8.355 мс
`par` (Швидкий предикат: a < b)	0.186 мс	0.194 мс	0.763 мс	0.756 мс	7.820 мс	7.703 мс
`par` (Важкий предикат: |a| < |b|)	0.258 мс	0.266 мс	1.004 мс	1.094 мс	10.598 мс	11.892 мс

Ключове спостереження: На платформі Intel i3-1005G1 послідовний виклик `std::min_element` (seq) досягає найкращого часу (2.056 мс для N = 10^7), оскільки операція є чисто Memory-bound, а овергед пулу потоків Intel TBB у політиках `par`/`par_unseq` перевищує обчислювальний час сканування.

3. ДОСЛІДЖЕННЯ ВЛАСНОГО ПАРАЛЕЛЬНОГО АЛГОРИТМУ ВІД ПАРАМЕТРА K

У власному алгоритмі масив розбивається на K частин одинакової довжини. Нижче наведено протокол вимірювань для масиву N = 10^7 елементів на процесорі Intel i3-1005G1 (`hardware_concurrency = 4`).

K частин	Час -O3 (мс)	Прискорення S(K) (-O3)	Час -O0 (мс)	Прискорення S(K) (-O0)	Ефективність E(K) (-O3)
1	49.025 мс	1.00x	46.652 мс	1.00x	100.0%
2	31.050 мс	1.58x	28.778 мс	1.62x	79.0%
4 (K_{opt})	**14.478 мс**	**3.39x**	**14.596 мс**	**3.20x**	**84.8%**
8	15.828 мс	3.10x	17.226 мс	2.71x	38.8%
12	19.028 мс	2.58x	16.383 мс	2.85x	21.5%
16	18.226 мс	2.69x	17.518 мс	2.66x	16.8%
24	18.320 мс	2.68x	18.951 мс	2.46x	11.2%
32	18.458 мс	2.66x	21.068 мс	2.21x	8.3%
64	19.801 мс	2.48x	41.193 мс	1.13x	3.9%
128	21.301 мс	2.30x	37.708 мс	1.24x	1.8%
256	28.190 мс	1.74x	51.099 мс	0.91x	0.7%

4. АНАЛІЗ

- **Співвідношення K_{opt} з апаратними потоками CPU:** Для великої вибірки (N = 10^7) оптимум досягається строго при **K_{opt} = 4** (час зменшується з 49.025 мс до 14.478 мс, прискорення 3.39x). Співвідношення з апаратною кількістю потоків процесора: K_{opt} `/ hardware_threads = 4 / 4 = 1.00`. Оскільки задача є обчислювально інтенсивною без блокувань введення-виведення, створення більшої кількості потоків, ніж 4, не збільшує фізичний паралелізм.
- **Закон зростання часу при K > K_{opt}:** При зростанні K від 4 до 256 час виконання монотонно зростає з 14.478 мс до 28.190 мс за квазілінійним законом. Це зумовлено: (1) системними витратами ОС на виділення пам'яті під стек кожного об'єкта `std::jthread`; (2) накладними витратами на перемикання контексту (Context Switching) на 2 фізичних ядрах; (3) вимиванням кеш-пам'яті L3 (4 MB).
- **Вплив оптимізації та складності предиката:** Впровадження важкого предиката (|a| < |b|) збільшує час виконання `par` з 7.703 мс до 11.892 мс через додаткові обчислювальні інструкції `std::abs`.

5. ВИСНОВКИ

Паралелізація власного алгоритму для N = 10^7 забезпечує прискорення 3.39x при K_{opt} = 4, що точно відповідає кількості апаратних потоків Intel i3-1005G1. Водночас для простих скалярних порівнянь послідовний виклик `std::min_element` виявився найшвидшим (2.056 мс) завдяки відсутності системного овергеду на синхронізацію потоків.